# 21-Day Python Programming Challenge

*From Fundamentals to Advanced - Units 3-6 + DSA*

## Introduction

Welcome to the 21-Day Python Programming Challenge! This comprehensive program is designed to take you from intermediate to advanced Python programming, covering Units 3-6 of your BCA course along with integrated Data Structures and Algorithms (DSA) problems.

### Challenge Structure:

- Days 1-7: Easy Level (Modules, Packages, Regular Expressions + Basic DSA)
- Days 8-14: Medium Level (File Handling, Exception Handling + Intermediate DSA)
- Days 15-21: Hard Level (OOP, NumPy, Pandas, Matplotlib + Advanced DSA)

**Note:** Even 'easy' questions require substantial implementation (not just 2-line solutions). Focus on writing clean, well-documented code with proper error handling.

## Day 1: Introduction to Modules

**Difficulty:** Easy | **Topic:** Creating and importing modules

**Learning Objectives:**

- Understand module creation and import mechanisms
- Learn different import syntaxes
- Implement basic utility modules

**Question 1: Custom Math Utilities Module**
Create a module named 'math_utils.py' that contains the following functions:

- is_prime(n): Check if a number is prime
- factorial(n): Calculate factorial using iteration
- gcd(a, b): Find greatest common divisor using Euclidean algorithm
- lcm(a, b): Find least common multiple
- is_perfect_number(n): Check if number equals sum of proper divisors

Create a main program that imports and tests all functions with user input. Include proper docstrings and input validation.

**Question 2: String Operations Module**
Create a 'string_ops.py' module with functions to: count vowels/consonants, reverse words in sentence, check palindrome (ignoring case/spaces), count word frequency in text, and convert between camel_case, snake_case, and PascalCase. Test with a variety of inputs.

**Question 3: Unit Converter Module**
Build a 'converter.py' module for temperature (Celsius, Fahrenheit, Kelvin), length (meters, feet, inches, kilometers), and weight (kg, pounds, grams) conversions. Include bidirectional conversion functions and a menu-driven interface.

## Day 2: Standard Library Modules

**Difficulty:** Easy | **Topic:** sys, os, math, time modules

**Learning Objectives:**

- Master Python standard library modules
- Work with system operations and file paths
- Implement time-based operations

**Question 1: System Information Reporter**
Using sys and os modules, create a program that displays: Python version, platform info, current working directory, environment variables, command-line arguments, and system path. Format the output in a readable report with proper sections.

**Question 2: Directory Navigator and Analyzer**
Create a program using os module that: lists all files/folders in given directory, displays file sizes in human-readable format (KB, MB, GB), counts files by extension, creates directory tree visualization, and calculates total space used. Handle permission errors gracefully.

**Question 3: Advanced Calculator with Math Module**
Build a scientific calculator using the math module with: trigonometric functions (sin, cos, tan with degree/radian modes), logarithmic functions (log, log10, ln), power and root operations, factorial and combinations/permutations, and statistical functions (mean, median, standard deviation). Include a user-friendly menu and input validation.

**Question 4: Execution Time Benchmarker**
Using the time module, create a benchmarking tool that: measures execution time of different sorting algorithms (bubble, selection, insertion), compares performance with different input sizes, displays results in formatted table, and generates time complexity analysis report. Test with random, sorted, and reverse-sorted arrays.

## Day 3: Packages and Package Management

**Difficulty:** Easy | **Topic:** Creating packages, __init__.py, package structure

**Question 1: Student Management Package**
Create a package 'student_mgmt' with modules: student.py (Student class with attributes), operations.py (add, delete, search students), reports.py (generate grade reports, calculate GPA), and database.py (save/load student data to JSON). Include __init__.py to expose main functionality.

**Question 2: Geometry Package**
Build a 'geometry' package with subpackages: shapes2d (circle, rectangle, triangle), shapes3d (sphere, cube, cylinder), each with area, perimeter/volume calculations. Create a calculator module that uses all shapes. Structure with proper __init__.py files.

**Question 3: Data Structures Package + DSA**
Create a 'ds_package' with implementations of: Stack (array-based and linked-list), Queue (circular queue), LinkedList (singly and doubly), and BinaryTree (basic

operations). Each in separate modules with comprehensive methods. Write test cases demonstrating each structure.

## Day 4: Regular Expressions Basics

**Difficulty:** Easy | **Topic:** Pattern matching, search, match, findall

**Question 1: Email Validator and Extractor**
Create a program that validates email addresses using regex patterns. Extract all emails from given text, categorize by domain (.com, .org, .edu), identify and separate valid/invalid emails, and generate statistics report. Test with various email formats.

**Question 2: Phone Number Formatter**
Build a tool that: recognizes phone numbers in different formats (with/without country code, with dashes, dots, spaces), standardizes them to common format, validates Indian mobile numbers, and extracts area codes. Handle international formats.

**Question 3: Text Pattern Analyzer**
Create a comprehensive text analyzer that: finds all dates in text (various formats), extracts URLs and categorizes them, identifies hashtags and mentions, finds repeated words, detects IP addresses, and generates pattern statistics report.

**Question 4: Password Strength Checker**
Using regex, build a password validator that: checks minimum length, requires uppercase, lowercase, digits, special characters, detects common patterns (repeated chars, sequential numbers), assigns strength score (weak/medium/strong), and suggests improvements for weak passwords.

## Day 5: Advanced Regular Expressions

**Difficulty:** Easy-Medium | **Topic:** Groups, meta characters, substitution

**Question 1: Log File Parser**
Create a program to parse server log files. Extract: timestamps, IP addresses, request methods, URLs, status codes, response sizes. Use groups to capture different components. Generate summary: most accessed pages, error rate, unique visitors, peak hours.

**Question 2: Code Comment Extractor**
Build a tool that extracts comments from Python source code: single-line (#), multi-line (''' or """), inline comments, docstrings. Preserve structure and line numbers. Count comment density, identify TODO/FIXME markers, and generate documentation skeleton.

**Question 3: Data Scrubber and Anonymizer**
Using re.sub(), create a data anonymizer that: replaces credit card numbers with masked version, anonymizes phone numbers (keep last 4 digits), redacts email addresses (keep domain), masks SSN, removes PII from text while maintaining readability. Handle multiple formats.

## Day 6: Basic DSA - Arrays and Strings

**Difficulty:** Easy-Medium | **Topic:** Array manipulation, string algorithms

**Question 1: Array Rotation and Manipulation**
Implement functions to: rotate array left/right by k positions (in-place), find maximum sum of contiguous subarray (Kadane's algorithm), rearrange array in alternating positive-negative, find leaders in array (element greater than all to its right), and move all zeros to end. Test with various arrays.

**Question 2: String Algorithms Suite**
Create implementations of: longest common subsequence between two strings, longest palindromic substring (efficient algorithm), string pattern matching (KMP or Boyer-Moore), check if strings are rotations of each other, and find all permutations of string. Compare time complexities.

**Question 3: Two Pointer Technique Problems**
Solve using two-pointer approach: find pair with given sum in sorted array, remove duplicates from sorted array (in-place), container with most water problem, 3-sum problem (find triplets with zero sum), and longest substring without repeating characters. Analyze time/space complexity.

## Day 7: Basic DSA - Searching and Sorting

**Difficulty:** Easy-Medium | **Topic:** Search algorithms, sorting techniques

**Question 1: Comprehensive Sorting Implementations**
Implement all major sorting algorithms: bubble sort (optimized), selection sort, insertion sort, merge sort, quick sort (with different pivot strategies), heap sort. Create visualization of step-by-step execution. Compare performance with different input sizes and types (random, sorted, reverse-sorted).

**Question 2: Binary Search Variations**
Implement: standard binary search, find first/last occurrence of element, count occurrences in sorted array, search in rotated sorted array, find peak element, and square root using binary search. Handle edge cases and analyze time complexity.

**Question 3: Search and Sort Applications**
Create a student database system that: stores student records with multiple attributes, implements binary search by ID/name, sorts by different criteria (GPA, name, age), finds k-th smallest/largest element, and performs range queries. Handle large datasets efficiently.

## Day 8: File Handling - Reading and Writing

**Difficulty:** Medium | **Topic:** File operations, text and binary files

**Question 1: Multi-Format File Converter**
Build a file converter that: reads CSV and converts to JSON/XML, converts JSON to CSV with proper handling of nested data, merges multiple text files with formatting, splits large file into smaller chunks with size limits, and handles different encodings (UTF-8, ASCII). Include progress indicators.

**Question 2: Configuration File Manager**
Create a configuration management system that: reads/writes INI and JSON config files, validates config values, supports nested configurations, handles comments in config files, implements config versioning, provides config diff functionality, and rollback to previous versions.

**Question 3: File Encryption/Decryption Tool**
Implement a file security tool that: encrypts/decrypts files using Caesar cipher and substitution cipher, handles binary files, implements password-based encryption, creates file checksums for integrity verification, and securely deletes original files. Include a user-friendly CLI.

## Day 9: Advanced File Operations

**Difficulty:** Medium | **Topic:** File seeking, binary files, with statement

**Question 1: Large File Processor**
Handle large files efficiently: read and process file in chunks without loading entire file, implement pagination for large text files, find and replace in large files using seek(), create file index for quick access to specific sections, and implement efficient searching in large log files.

**Question 2: Binary File Manager**
Work with binary files: serialize/deserialize Python objects using pickle, create custom binary file format for student records, implement file compression/decompression, handle endianness in binary data, and create a simple database using binary files with indexing.

**Question 3: File System Operations Suite**
Build a file system utility: recursive directory traversal with pattern matching, file/folder size calculator with disk usage analysis, duplicate file finder using hash comparison, file backup system with incremental backups, and file synchronization between directories. Use with statement for safe file handling.

## Day 10: Exception Handling Fundamentals

**Difficulty:** Medium | **Topic:** try, except, else, finally blocks

**Question 1: Robust Calculator with Exception Handling**
Create a calculator that handles: division by zero, invalid input types, overflow errors, mathematical domain errors (sqrt of negative), and invalid operations. Use try-except-else-finally blocks appropriately. Log all errors with timestamps and provide user-friendly error messages.

**Question 2: File Operations with Complete Error Handling**
Build a file processor that handles: FileNotFoundError, PermissionError, IOError, UnicodeDecodeError, and disk full scenarios. Implement automatic retry logic, fallback mechanisms, detailed error logging, and graceful degradation. Ensure file handles are always closed using finally or with statement.

**Question 3: Data Validator with Multiple Exception Types**

Create a data validation system that: validates user registration (age, email, phone), handles ValueError, TypeError, AttributeError, KeyError for missing fields, implements custom validation exceptions, provides detailed error messages with suggestions, and maintains validation log. Use exception chaining where appropriate.

## Day 11: Custom Exceptions and raise

**Difficulty:** Medium | **Topic:** Custom exception classes, raise, assert

**Question 1: Banking System with Custom Exceptions**
Design a banking system with custom exceptions: InsufficientFundsError, InvalidAccountError, TransactionLimitExceeded, NegativeAmountError. Implement deposit, withdrawal, transfer operations with proper exception raising. Use assert for preconditions. Include transaction logging and rollback mechanisms.

**Question 2: E-Commerce Inventory System**
Build inventory manager with custom exceptions: OutOfStockError, InvalidPriceError, DuplicateProductError, InvalidQuantityError. Implement add/remove/update inventory, process orders, handle stock alerts. Use exception hierarchy and exception chaining. Include comprehensive testing of all exception scenarios.

**Question 3: API Response Handler**
Create an API simulator with custom exceptions: APITimeoutError, InvalidAPIKeyError, RateLimitExceeded, InvalidResponseFormat. Simulate various API states, implement retry logic with exponential backoff, handle partial failures, and create comprehensive error reports. Use raise from for exception chaining.

## Day 12: Stack and Queue Implementations

**Difficulty:** Medium | **Topic:** DSA - Stacks and Queues

**Question 1: Expression Evaluator Using Stack**
Implement: infix to postfix conversion, postfix expression evaluation, balanced parentheses checker (supporting multiple types: (), [], {}), implement next greater element, and create a min stack (O(1) for min operation). Handle operator precedence and associativity correctly.

**Question 2: Queue Applications**
Implement: circular queue with fixed size, deque (double-ended queue) with all operations, priority queue using heap, implement queue using two stacks, and create a sliding window maximum using deque. Test with various scenarios and edge cases.

**Question 3: Task Scheduler Using Queues**
Build a task scheduling system: implement priority queue for tasks, handle task dependencies, implement round-robin scheduling, create waiting queue and execution queue, handle task timeouts and failures, and generate execution timeline. Support task preemption and resumption.

## Day 13: Linked Lists and Trees

**Difficulty:** Medium-Hard | **Topic:** DSA - Advanced data structures

**Question 1: Advanced Linked List Operations**
Implement singly and doubly linked lists with: detect and remove cycle, find middle element, reverse linked list (iterative and recursive), merge two sorted lists, check if palindrome, and clone linked list with random pointers. Analyze time/space complexity for each.

**Question 2: Binary Tree Implementations**
Create binary tree with: all traversals (inorder, preorder, postorder, level-order), find height and diameter, check if balanced, lowest common ancestor, path sum problems, serialize/deserialize tree, and mirror tree. Implement both recursive and iterative approaches.

**Question 3: Binary Search Tree Operations**
Build BST with: insert, delete, search operations, find kth smallest/largest element, validate BST, convert sorted array to BST, find inorder successor/predecessor, and range queries. Implement self-balancing using AVL rotations (optional challenge).

## Day 14: Hashing and Hash Tables

**Difficulty:** Medium-Hard | **Topic:** DSA - Hash maps and applications

**Question 1: Hash Table Implementation**
Implement hash table from scratch: design hash function, handle collisions using chaining and open addressing, implement rehashing when load factor exceeds threshold, support get/put/delete operations, and compare performance with Python dict. Test with large datasets.

**Question 2: Hash-Based Problem Solving**
Solve using hashing: find first non-repeating character, group anagrams together, implement LRU cache, find longest consecutive sequence, two sum and its variations, subarray sum equals k, and find all pairs with given difference. Optimize for time complexity.

**Question 3: Frequency and Counting Problems**
Create solutions using hash maps: find top k frequent elements, implement word frequency counter with ranking, find majority element (Boyer-Moore voting), detect duplicates within k distance, and group elements by frequency. Include analysis of hash collision scenarios.

## Day 15: Object-Oriented Programming - Classes

**Difficulty:** Medium-Hard | **Topic:** Classes, objects, constructors, methods

**Question 1: Library Management System**
Design classes for: Book (title, author, ISBN, status), Member (name, ID, borrowed books), Library (catalog, issue/return books, search). Implement: due date tracking, fine calculation, reservation system, member history, and book recommendations. Use class/instance variables appropriately.

**Question 2: Banking System with OOP**

Create Account class hierarchy: SavingsAccount, CurrentAccount, FixedDeposit. Implement: interest calculation, minimum balance maintenance, transaction limits, account statements, inter-account transfers. Include Customer class with multiple accounts. Use private attributes and property decorators.

**Question 3: E-Commerce System**
Design: Product, Order, Customer, ShoppingCart, Payment classes. Implement: add/remove from cart, apply discounts/coupons, calculate taxes and shipping, order tracking, inventory management, and customer reviews. Use special methods (__str__, __repr__, __eq__) appropriately.

## Day 16: Inheritance and Polymorphism

**Difficulty:** Hard | **Topic:** Inheritance types, method overriding, polymorphism

**Question 1: Shape Hierarchy with Polymorphism**
Create Shape base class with derived classes: Circle, Rectangle, Triangle, Polygon. Implement polymorphic methods: area(), perimeter(), draw(). Add 3D shapes: Sphere, Cube, Cylinder with volume(). Demonstrate method resolution order (MRO) and super(). Create a drawing canvas that handles all shapes uniformly.

**Question 2: Employee Management with Multiple Inheritance**
Design: Employee base class, Developer, Manager, SalesPerson subclasses. Implement multiple inheritance for ManagerDeveloper. Add: salary calculation (different for each type), performance evaluation, promotion system, benefits calculation. Use abstract base class for common interface. Demonstrate diamond problem resolution.

**Question 3: Vehicle Rental System**
Create Vehicle base class with Car, Bike, Truck subclasses. Implement: rental pricing (varies by type and duration), maintenance tracking, availability checking, insurance calculation. Add ElectricVehicle and HybridVehicle using multiple inheritance. Include operator overloading for vehicle comparison by rental price.

## Day 17: Advanced OOP Concepts

**Difficulty:** Hard | **Topic:** Abstract classes, operator overloading, decorators

**Question 1: Custom Data Structures with Operator Overloading**
Implement Vector and Matrix classes with operator overloading: +, -, *, / for arithmetic, ==, !=, <, > for comparison, [] for indexing, len() support. Add dot product, cross product, matrix transpose, determinant. Include comprehensive error handling and type checking.

**Question 2: Design Patterns Implementation**
Implement design patterns: Singleton (database connection), Factory (object creation), Observer (event notification system), Strategy (payment methods). Use abstract base classes where appropriate. Demonstrate each pattern with real-world example and explain when to use each.

**Question 3: Custom Container Class**

Create a custom collection class that: supports iteration (__iter__, __next__), indexing and slicing, context management (__enter__, __exit__), comparison operators, and arithmetic operations. Implement as a generic container with type hints. Add methods for common operations (filter, map, reduce).

## Day 18: NumPy Fundamentals

**Difficulty:** Hard | **Topic:** NumPy arrays, operations, broadcasting

**Question 1: NumPy Array Manipulations**
Create programs using NumPy: generate matrices using arange, linspace, zeros, ones, random. Perform reshape, transpose, flatten operations. Implement matrix operations: multiplication, inverse, eigenvalues. Solve system of linear equations. Compare performance with Python lists for large datasets.

**Question 2: Statistical Analysis with NumPy**
Build statistical analyzer: calculate mean, median, mode, standard deviation, variance, percentiles, correlation coefficient, covariance matrix. Implement moving averages, cumulative sums, normalize data (z-score, min-max). Handle missing values. Generate random data from normal, uniform, poisson distributions.

**Question 3: Image Processing with NumPy**
Represent images as NumPy arrays: convert to grayscale, apply filters (blur, sharpen, edge detection), rotate and flip images, adjust brightness and contrast, create image histograms, implement basic convolution operations. Work with RGB channels separately. Compare original and processed images.

## Day 19: Pandas Data Manipulation

**Difficulty:** Hard | **Topic:** DataFrames, Series, data operations

**Question 1: Data Cleaning and Preparation**
Load dataset from CSV: handle missing values (drop, fill with mean/median/mode), remove duplicates, detect and handle outliers, convert data types, standardize column names, merge multiple datasets, handle date-time columns, create derived columns. Generate data quality report.

**Question 2: Advanced Data Analysis**
Perform analysis on sales dataset: group by multiple columns, apply aggregation functions (sum, mean, count), pivot tables, cross-tabulation, time series analysis (resampling, rolling windows), correlation analysis, filtering with multiple conditions. Create summary statistics for each category.

**Question 3: Student Performance Analyzer**
Create comprehensive student analysis system: load data from multiple sources (CSV, Excel), calculate grades and percentages, rank students overall and subject-wise, identify top performers and struggling students, generate subject-wise statistics, create grade distribution, export reports to multiple formats. Include data validation and error handling.

## Day 20: Data Visualization with Matplotlib

**Difficulty:** Hard | **Topic:** Plotting, customization, subplots

**Question 1: Comprehensive Plotting Suite**
Create various plots: line plots with multiple series, bar charts (grouped, stacked), histograms with customizable bins, scatter plots with regression lines, pie charts with percentages, box plots for distribution analysis. Customize: colors, labels, legends, titles, grid, axis scales (linear, log). Save in multiple formats.

**Question 2: Time Series Visualization**
Visualize time series data: plot trends over time, add moving averages, show seasonal patterns, highlight anomalies, compare multiple time series, create area charts for cumulative data. Add annotations for important events. Implement interactive features (zooming, panning). Handle missing data gracefully.

**Question 3: Dashboard with Multiple Subplots**
Create analytical dashboard using subplots: combine different plot types (line, bar, scatter, heatmap) in one figure, create correlation matrices, visualize geographical data, implement small multiples for category comparison. Add proper spacing, shared axes where appropriate, and consistent styling. Include summary statistics in text boxes.

## Day 21: Capstone Project - Integrated Application

**Difficulty:** Hard | **Topic:** Comprehensive integration of all concepts

**Final Challenge: Complete Data Analysis and Visualization System**
Build a comprehensive system that integrates all concepts learned:

1. **Module Structure:**
- Create proper package structure with multiple modules
- Use custom exceptions for error handling
- Implement logging throughout the application

2. **Data Management:**
- Read data from multiple file formats (CSV, JSON, Excel)
- Clean and preprocess data using Pandas
- Handle missing values and outliers appropriately
- Validate data using regular expressions

3. **OOP Implementation:**
- Create class hierarchy for data models
- Implement inheritance and polymorphism
- Use abstract base classes for interfaces
- Implement design patterns (Factory, Singleton, Observer)

4. **Data Analysis with NumPy/Pandas:**
- Perform statistical analysis
- Implement data transformations and aggregations
- Create pivot tables and cross-tabulations
- Generate summary reports

5. **Visualization with Matplotlib:**
- Create multiple plot types (line, bar, scatter, heatmap)
- Build interactive dashboard with subplots
- Customize all visual elements
- Export visualizations in multiple formats

6. **DSA Implementation:**
- Use appropriate data structures (hash maps, trees, heaps)
- Implement efficient search and sort algorithms
- Optimize for time and space complexity

7. **Additional Requirements:**
- Comprehensive error handling with custom exceptions
- Configuration file management (JSON/INI)
- Command-line interface with argparse
- Complete documentation (docstrings, README)
- Unit tests for critical functions
- Performance profiling and optimization

**Suggested Project Ideas:**

- Student Performance Analytics System
- Stock Market Analysis and Visualization Tool
- COVID-19 Data Tracker and Predictor
- E-Commerce Sales Analytics Dashboard
- Weather Data Analysis and Forecasting System

## Tips for Success

8. Practice daily - consistency is key
9. Write clean, well-documented code from day 1
10. Test your code with edge cases
11. Focus on understanding concepts, not just syntax
12. Build progressively - each day builds on previous days
13. Review and refactor your code regularly
14. Don't skip the 'easy' days - they build strong foundations
15. Use proper version control (Git) from the start

## Good luck with your 21-day challenge!

*Remember: The journey of a thousand miles begins with a single line of code.*